# Implementación de ModSecurity como WAF en Apache

# 1. Introducción

En el panorama actual de la ciberseguridad, las aplicaciones web representan uno de los principales vectores de ataque. Amenazas como la inyección SQL, el Cross-Site Scripting (XSS) o el uso de bots automatizados pueden comprometer la confidencialidad, integridad y disponibilidad de los sistemas. Por ello, además de aplicar buenas prácticas de desarrollo seguro, es esencial incorporar mecanismos de protección a nivel de servidor.

# 2. Entorno de trabajo

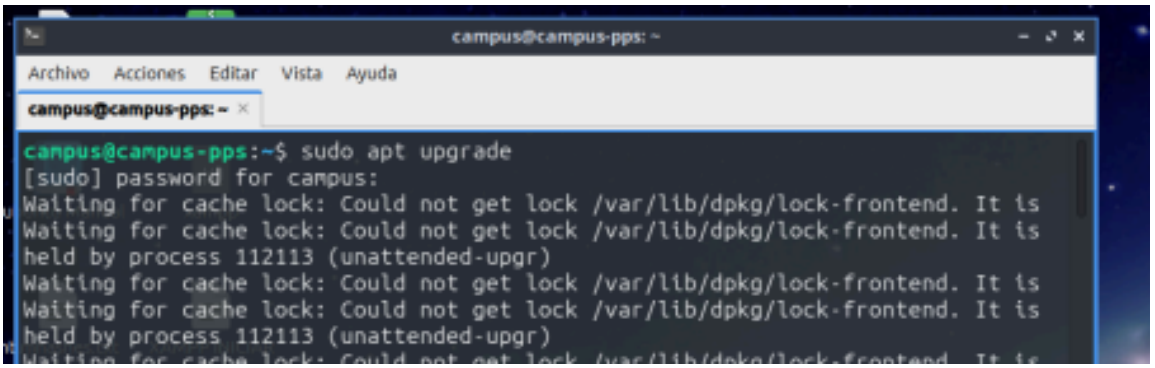
**Sistema operativo:** Linux (Ubuntu/Debian)
**Servidor web:** Apache 2
**WAF:** ModSecurity v2
**Reglas de seguridad:** OWASP ModSecurity Core Rule Set (CRS)

# 3. Instalación y configuración

## 3.1 Actualización del sistema

Para garantizar que todos los paquetes estuvieran actualizados, se ejecutaron los siguientes comandos:

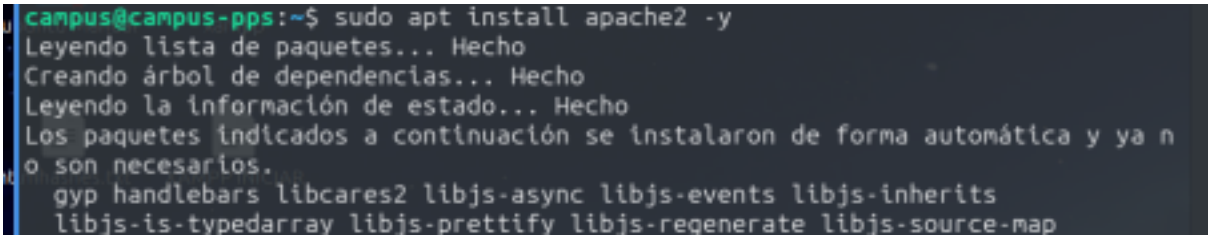
**sudo apt update**
**sudo apt upgrade -y**




## 3.2 Instalación de Apache

Se instaló el servidor web Apache en caso de no estar previamente instalado:

**sudo apt install apache2 -y**



### 3.3 Instalación de ModSecurity

A continuación, se instaló el módulo ModSecurity para Apache:

**sudo apt install libapache2-mod-security2 -y**

```
campus@campus-pps:~$ sudo apt install libapache2-mod-security2 -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya n
o son necesarios.
  gyp handlebars libcares2 libjs-async libjs-events libjs-inherits
  libjs-is-typedarray libjs-prettify libjs-regenerate libjs-source-map
```

### 3.4 Activación del módulo

Se activó el módulo de seguridad y se reinició Apache:

**sudo a2enmod security2**
**sudo systemctl restart apache2**

```
campus@campus-pps:~$ sudo a2enmod security2
Considering dependency unique_id for security2:
Module unique_id already enabled
Module security2 already enabled
campus@campus-pps:~$
```

```
campus@campus-pps:~$ sudo systemctl restart apache2
campus@campus-pps:~$
```

Para verificar que el módulo estaba cargado correctamente:

**sudo apachectl -M | grep security2**

```
campus@campus-pps:~$ sudo apachectl -M | grep security2
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using
127.0.1.1. Set the 'ServerName' directive globally to suppress this message
 security2_module (shared)
campus@campus-pps:~$
```

La salida confirmó que security2_module estaba activo.

## 4. Configuración de ModSecurity

ModSecurity se instala por defecto en modo de solo detección. Para activar el modo de protección:

**sudo cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf**

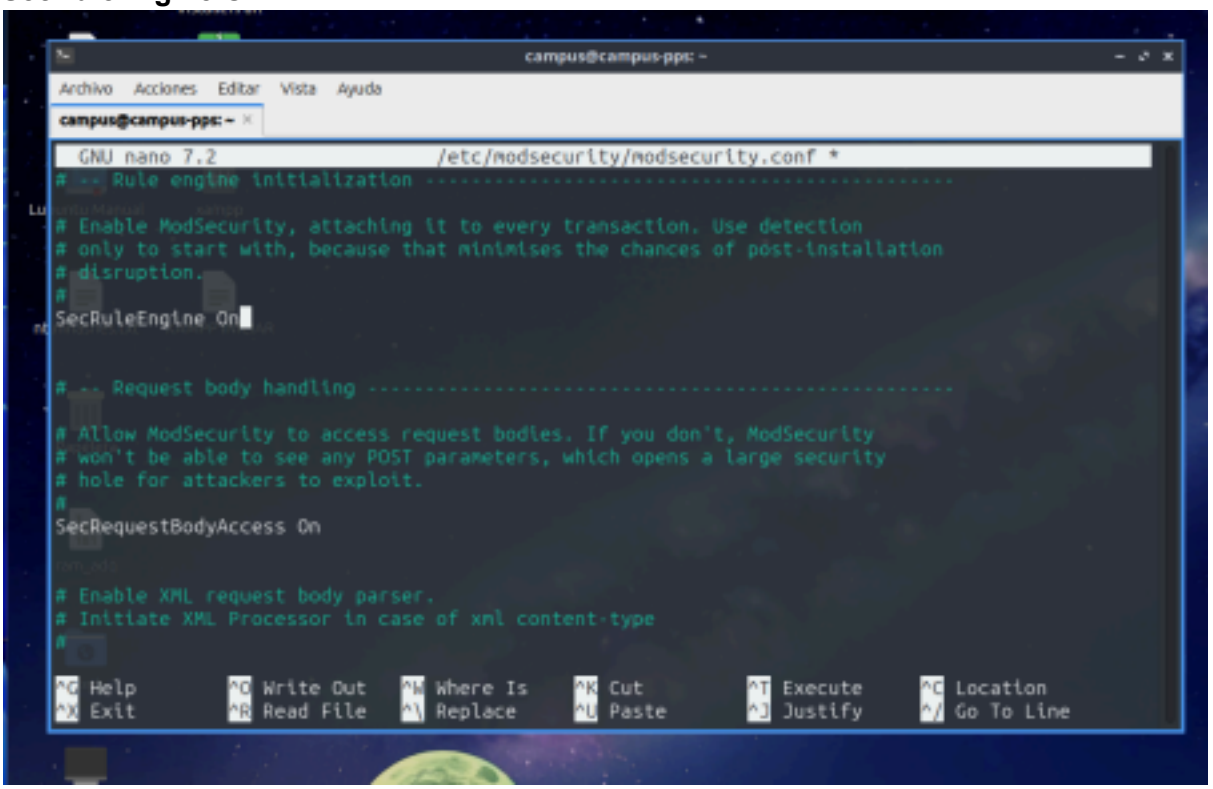
```
campus@campus-pps:~$ sudo cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/mod
security.conf
campus@campus-pps:~$
```

Se editó el archivo:
**sudo nano /etc/modsecurity/modsecurity.conf**

Y se modificó la siguiente línea:

**SecRuleEngine DetectionOnly**

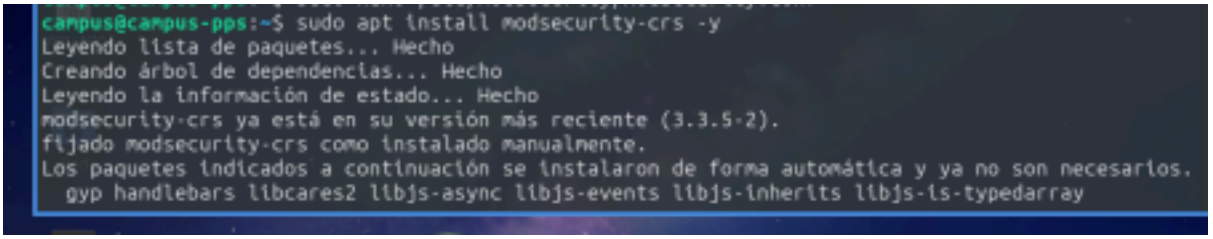
**SecRuleEngine On**




Esto permite que ModSecurity no solo detecte, sino que también **bloquee** las peticiones maliciosas.

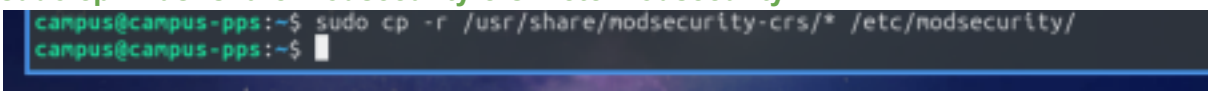
# 5. Instalación y activación de reglas OWASP CRS

**sudo apt install modsecurity-crs -y**



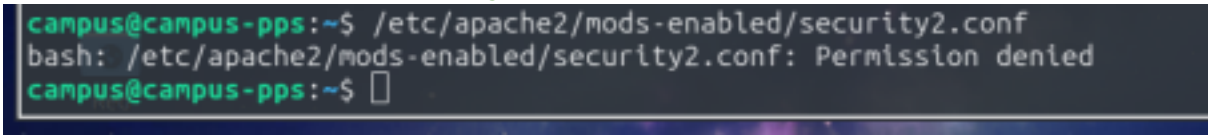

Las reglas se copiaron al directorio de configuración:

**sudo cp -r /usr/share/modsecurity-crs/* /etc/modsecurity/**




Se comprobó que Apache incluyera estas reglas en el archivo:

**/etc/apache2/mods-enabled/security2.conf**




Con las directivas:

**IncludeOptional /etc/modsecurity/*.conf**
**IncludeOptional /usr/share/modsecurity-crs/*.load**

```
IncludeOptional /etc/modsecurity/*.conf

# Include OWASP ModSecurity CRS rules if installed
IncludeOptional /usr/share/modsecurity-crs/*.load
```

Finalmente, se reinició Apache:

**sudo systemctl restart apache2**

```
campus@campus-pps:~$ sudo systemctl restart apache2
campus@campus-pps:~$
```

# 6. Pruebas y verificación del WAF

## 6.1 Prueba con User-Agent sospechoso

Se realizó una prueba simulando una herramienta de escaneo **curl -A "WPScan" http://localhost**

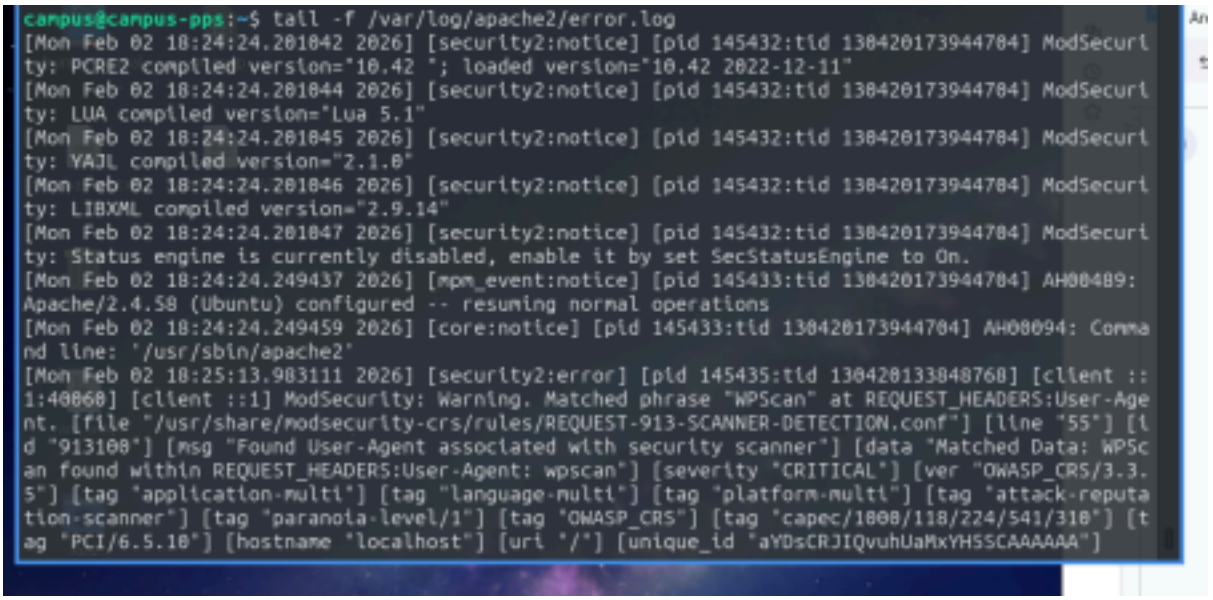
```
campus@campus-pps:~$ curl -A "WPScan" http://localhost
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at localhost Port 80</address>
</body></html>
campus@campus-pps:~$
```

**ModSecurity bloqueó o registró la petición, ya que WPScan es una herramienta de auditoría de WordPress cuyo User-Agent está contemplado en las reglas del CRS.**

## 6.2 Monitorización de logs

Para comprobar los eventos detectados:

tail -f /var/log/apache2/error.log




En los registros se visualizaron alertas con información sobre la regla activada y la petición bloqueada.

### 6.3 Prueba de ataque XSS

Se probó un intento de inyección de JavaScript:

bash

**curl -X GET "http://localhost/?test=<script>alert('XSS')</script>"**

ModSecurity detectó y bloqueó el ataque, demostrando su eficacia frente a amenazas del lado del cliente.

## 7. Inspección de reglas

Las reglas de ModSecurity pueden revisarse en:

/etc/modsecurity/rules/

Estas cubren ataques como:

- Inyección SQL
- XSS
- Fuerza bruta y bots
- Manipulación de cabeceras HTTP

## 8. Relación con los criterios de evaluación

**RA3.b:** Se han identificado riesgos de inyección mediante pruebas de XSS y escaneo

automatizado.

**RA3.f:** Se ha configurado Apache con ModSecurity y las reglas OWASP para mitigar ataques conocidos.

**RA3.g:** Las reglas CRS permiten detectar bots y ataques automatizados, reduciendo riesgos de fuerza bruta y abuso de formularios.

# 9. Conclusión

La implementación de ModSecurity como WAF en Apache proporciona una capa adicional de protección frente a ataques web comunes. Gracias al uso del OWASP Core Rule Set, es posible detectar y bloquear amenazas sin necesidad de modificar el código de la aplicación. Esta práctica cumple con los objetivos del RA3 y contribuye a reforzar la seguridad de los servicios web.